ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ

ΣΧΟΛΗ ΗΜΜΥ

Τηλεπικοινωνιακά Συστήματα Ι

Παράδωση 2ης εργασίας

Ημερομηνία Παράδοσης: 25/4/2026

Ομάδα 101

Κύρκος Κωνσταντίνος 2022030112

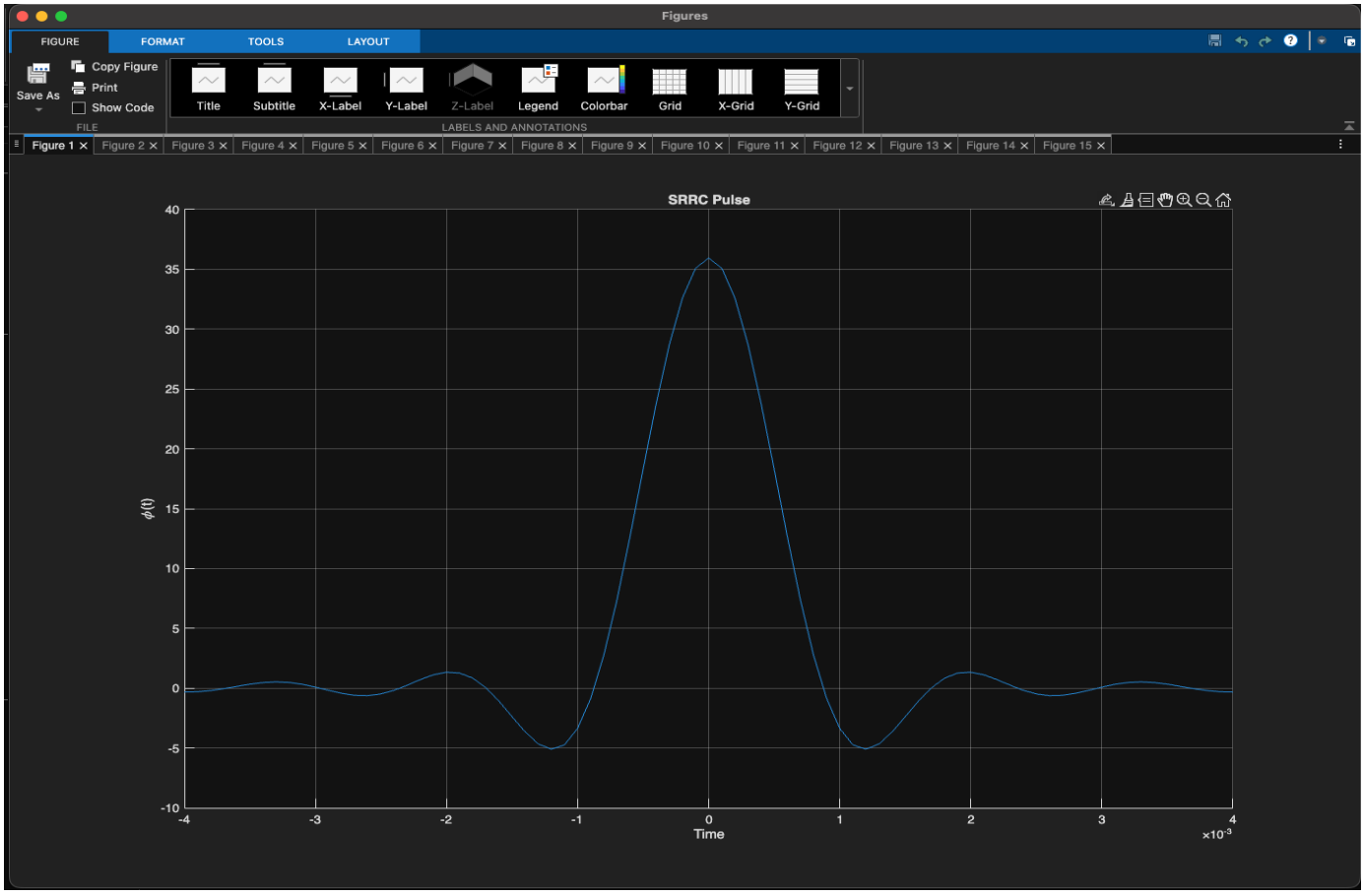
Ώρες απασχόλησης: 16

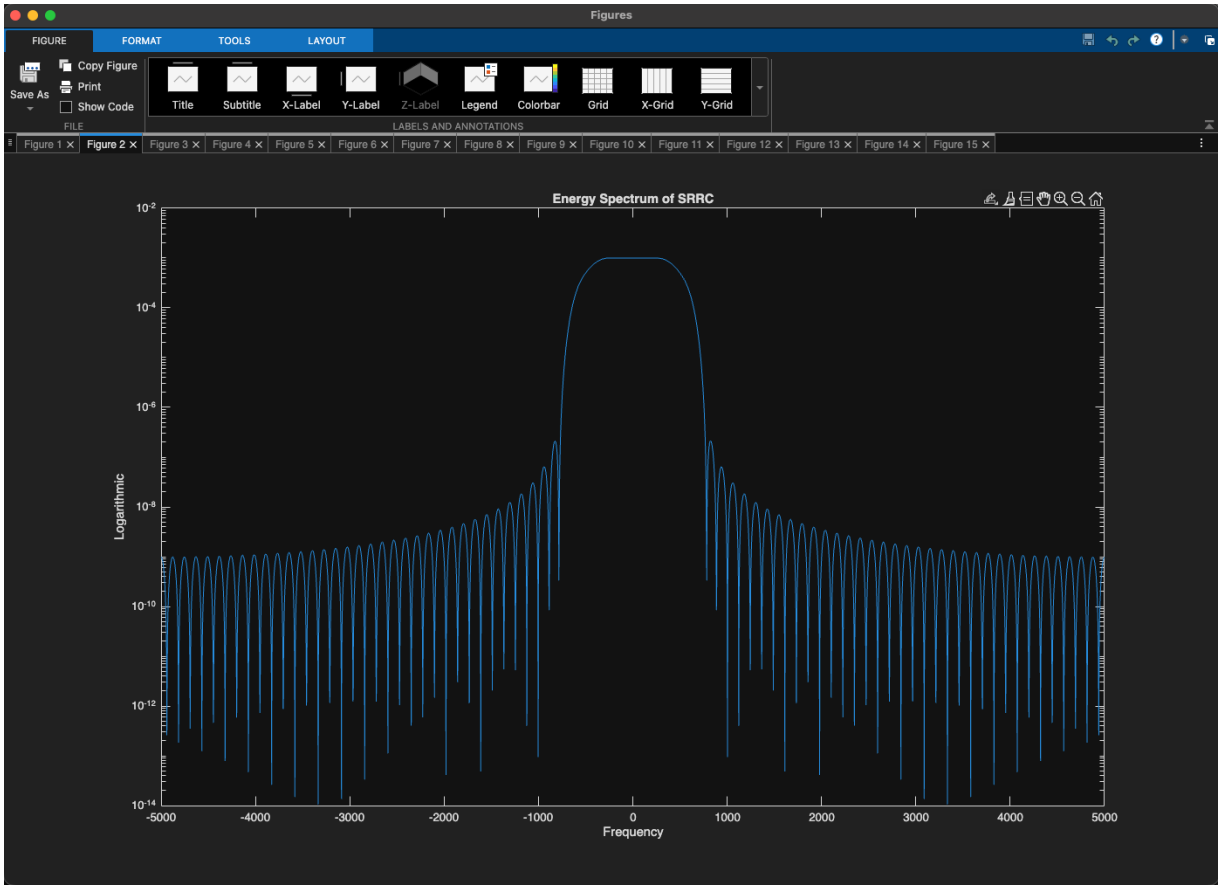
A.1

Construct an SRRC pulse ϕ(t) with parameters T = 10−3sec, over = 10, Ts = T over , A=4, and a =0.5. (10) Using the functions fftshift and fft, compute the magnitude of the Fourier transform of ϕ(t), |Φ(F)|, at Nf equally spaced points in the interval −Fs 2 , Fs 2 .1 Plot the energy spectral density |Φ(F)|^2 versus the appropriate frequency axis using semilogy.

Solution:

For the first step, we generated the SRRC pulse using the srrc_pulse.m function with the given parameters. We then computed its energy spectral density, using the fft and fft shift functions. A key detail here was setting the number of frequency points to Nf = 4096. We intentionally chose this large value to ensure it exceeds the length of our time-domain signals, thus avoiding any aliasing or distortion in our frequency plots. Finally, we plotted the spectrum using semilogy.





Code:

/Users/kyrkos/Documents/MATLAB/MATLAB_LAB2/untitled.m

```
clear all;
close all;
%% Variable decleration
T = 0.001;
over = 10;
Ts = T / over;
Fs = 1 / Ts;
A = 4;
a = 0.5;
K = 500;
N = 100;
Nf = 4096;

%% A.1

grid on;
hold on;
[phi, t] = srrc_pulse(T, over, A, a);
plot(t,phi);
xlim([-A*T A*T]);
title("SRRC Pulse");
xlabel("Time");
ylabel("\phi(t)");
hold off;

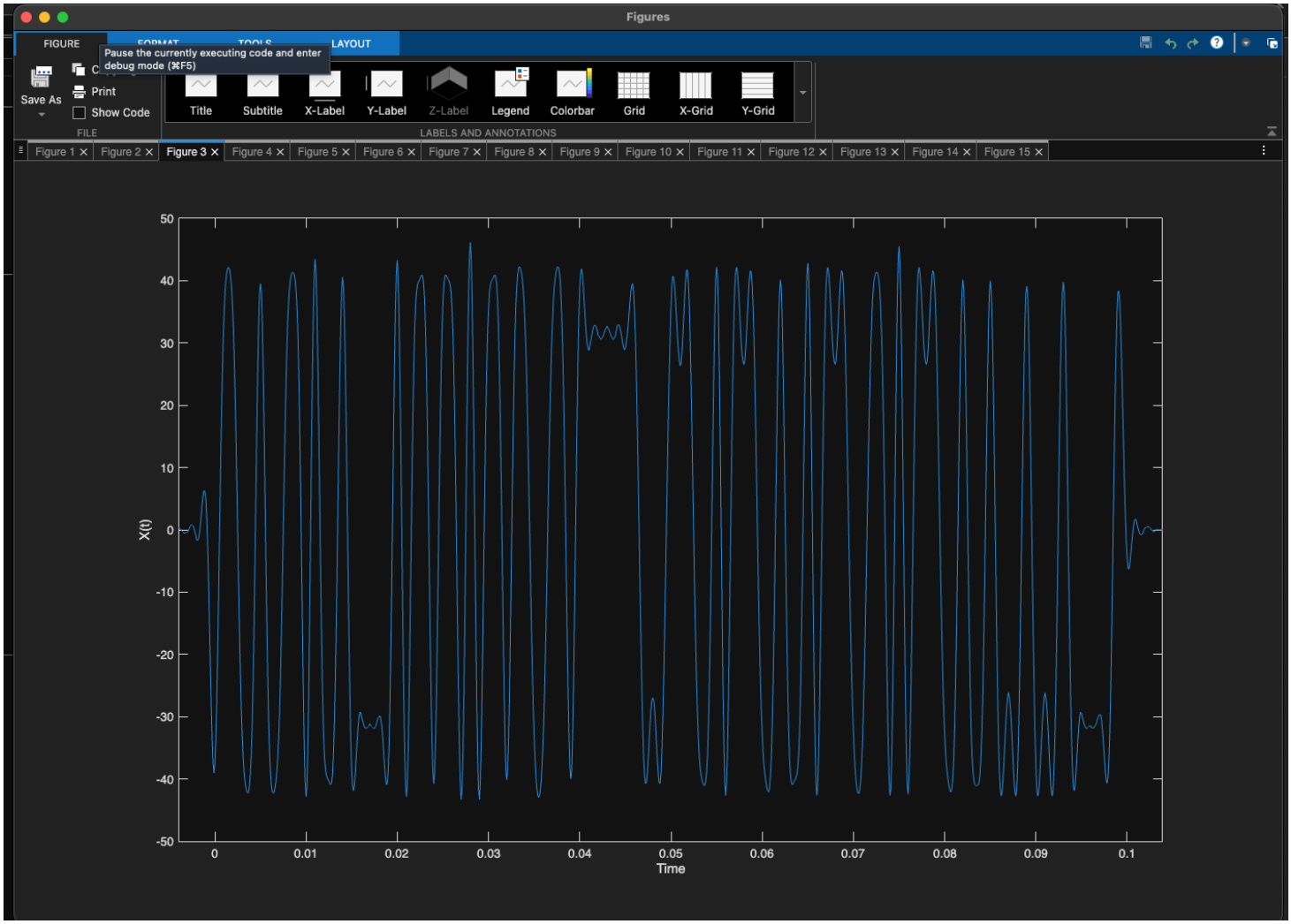
% Freq Range
f_axis = linspace(-Fs/2,(Fs/2-Fs/Nf), Nf);
% Fourier
phi_F = fftshift(fft(phi,Nf)*Ts);
abs_phi_F = abs(phi_F);
phi_Energy = power(abs_phi_F,2);
figure;
semilogy(f_axis, phi_Energy);
title("Energy Spectrum of SRRC");
xlabel("Frequency");
ylabel("Logarithmic");
xlim([-Fs/2 Fs/2]);

```

A.2

Solution:

Initially, a sequence of independent and equiprobable bits was generated. Using the custom bits_to_2PAM.m function, these bits were mapped to 2-PAM symbols, applying the standard rule of 0 to +1 and 1 to -1. To construct the continuous-time baseband waveform X(t) , the discrete symbol sequence was upsampled to match the system's sampling rate and then convolved with the SRRC pulse φ(t) generated in the previous step.



Code:

```
%% A.2

% 2-PAM bits

b = (sign(randn(N, 1)) + 1)/2;
x = bits_to_2PAM(b);

% Upsample the 2-PAM bit vector
upsample_x = (1/Ts)*upsample(x, over);
upsample_x_time = 0:Ts:N*T-Ts;

% Convolution of X and phi to compute the sum
X = conv(upsample_x,phi)*Ts;
X_time = upsample_x_time(1)+t(1):Ts:upsample_x_time(end)+t(end);
figure;
plot(X_time, X);
xlabel("Time");
ylabel("X(t)");
xlim([X_time(1) X_time(end)]);

```

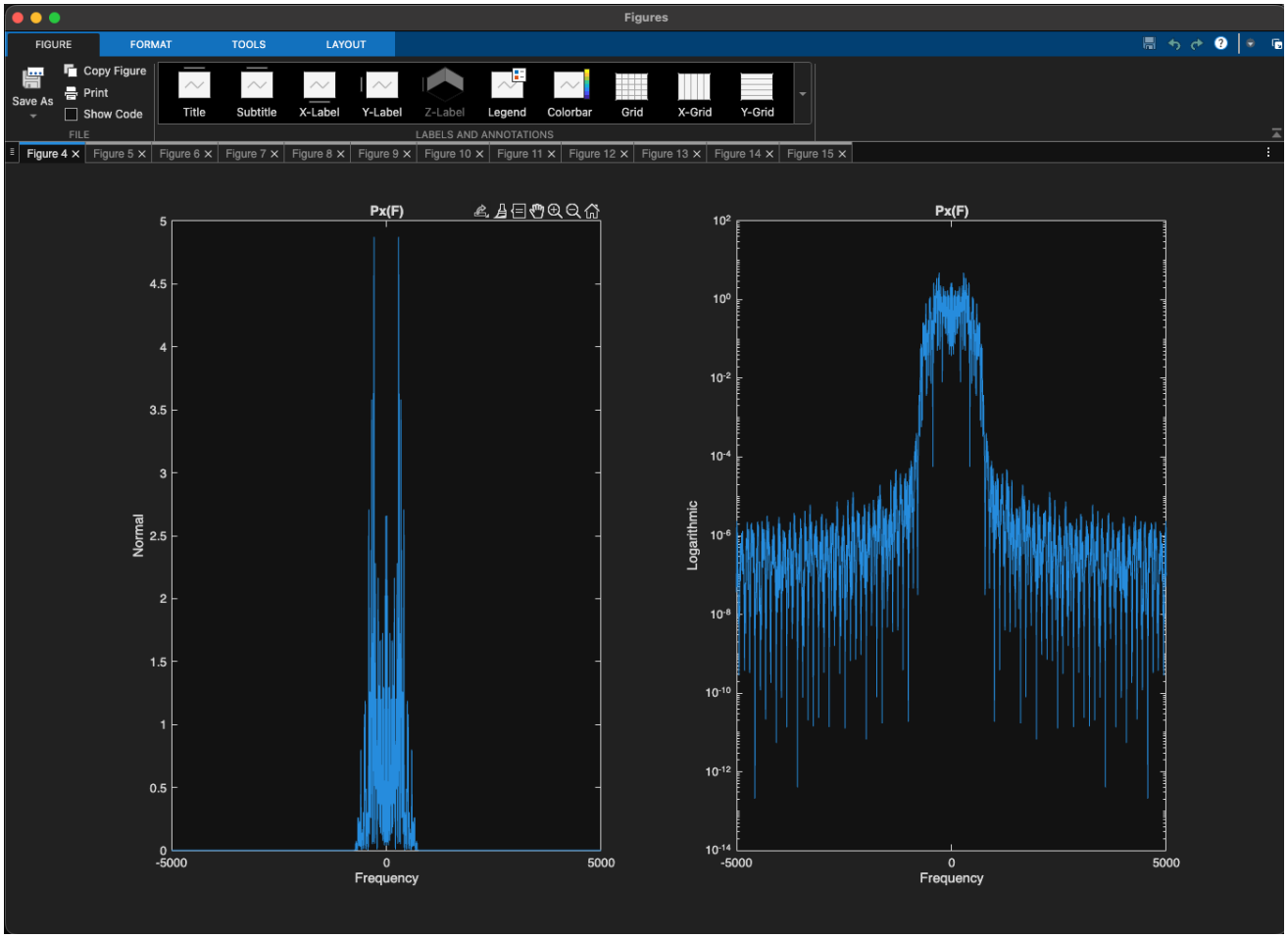
Assuming an infinite number of symbols, we proved that the power spectral density of X(t) is

$$S_X(F) = \frac{\sigma_X^2}{T} |\Phi(F)|^2 .$$

A.3

(10) Using the functions fft and fftshift, compute the periodogram of one realization of X(t): PX(F). Plot PX(F) using both plot and semilogy. Repeat for several different realizations of the bit sequence {b0,...,bN−1}, so as to get a good sense of what the periodograms of realizations of X(t) look like.

Solution:



In the above graph we can see the periodogram in plot and seminology.

Code:

```
%% A.3

% Create the periodogram
X_F = fftshift(fft(X, Nf)*Ts);
num = power(abs(X_F),2);
T_total = X_time(end) - X_time(1);
Px_F = num/T_total;

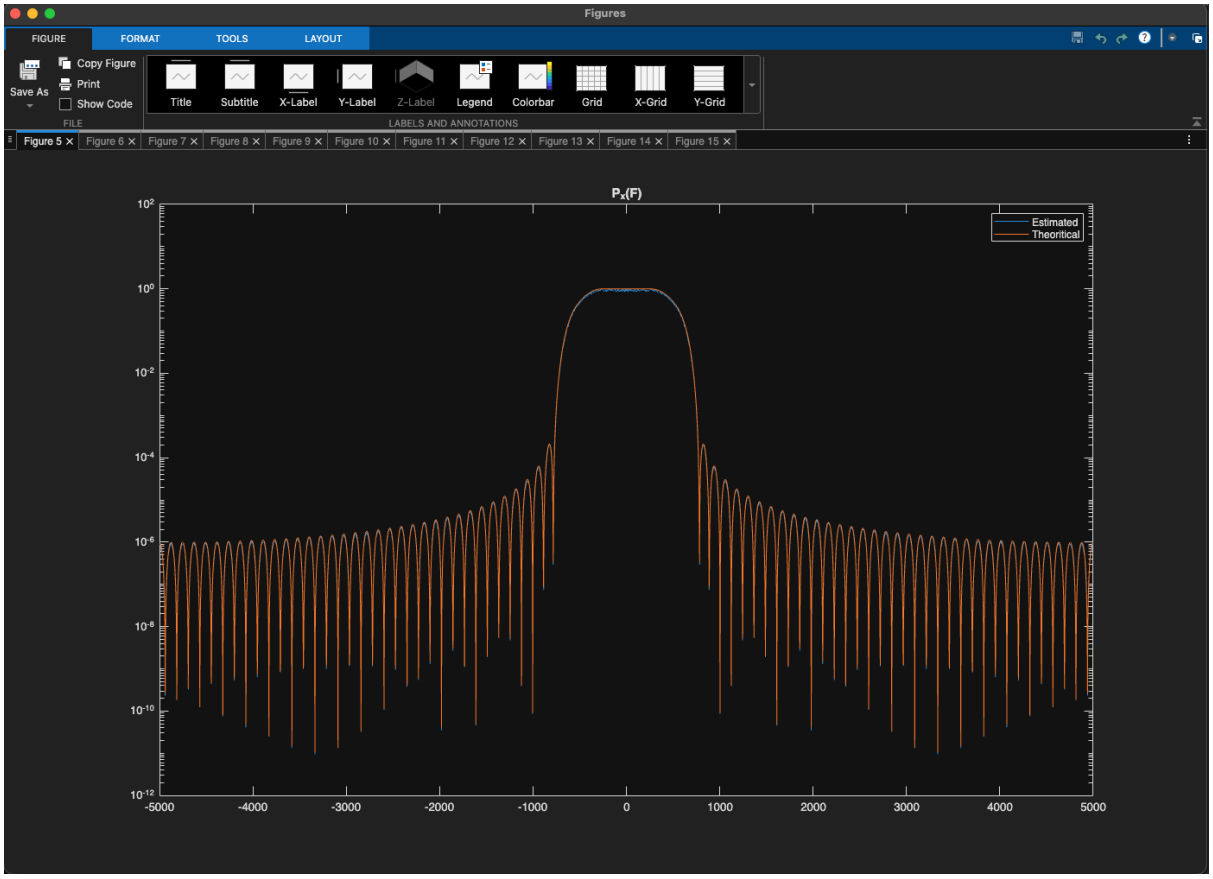
% plot and seminology
figure;
subplot(1,2,1);
plot(f_axis, Px_F);
title("Px(F)");
xlabel("Frequency");
ylabel("Normal");
xlim([-Fs/2 Fs/2]);

subplot(1,2,2);
semilogy(f_axis,Px_F);
title("Px(F)");
xlabel("Frequency");
ylabel("Logarithmic");
xlim([-Fs/2 Fs/2]);
```

(10) Estimate the power spectral density by computing numerical averages over K (indicatively, K = 500) periodogram realizations. On the same semilogy plot, show both the estimate and the theoretical power spectral density.

Solution:

Next, we needed to estimate the Power Spectral Density of our signal. Since the periodogram of a single realization is incredibly noisy, we computed numerical averages over K = 500 different periodogram realizations. We then plotted this experimental average alongside the theoretical PSD on the same semilogy plot. As expected, by averaging 500 realizations, the noise smoothed out drastically, and our estimated curve matched the theoretical line almost perfectly.



Code:

```
Px_F_total = 0;
for i=1:K
    % 2-PAM bits generator
    b = (sign(randn(N, 1)) + 1)/2;
    x = bits_to_2PAM(b);

    % Upsample the 2-PAM bit vector
    upsample_x = (1/Ts)*upsample(x, over);
    %upsample_x_time = 0:Ts:N*T-Ts;

    % Convolution of X and phi to compute the sum
    X = conv(upsample_x,phi)*Ts;
    %X_time = upsample_x_time(1)+t(1):Ts:upsample_x_time(end)+t(end);

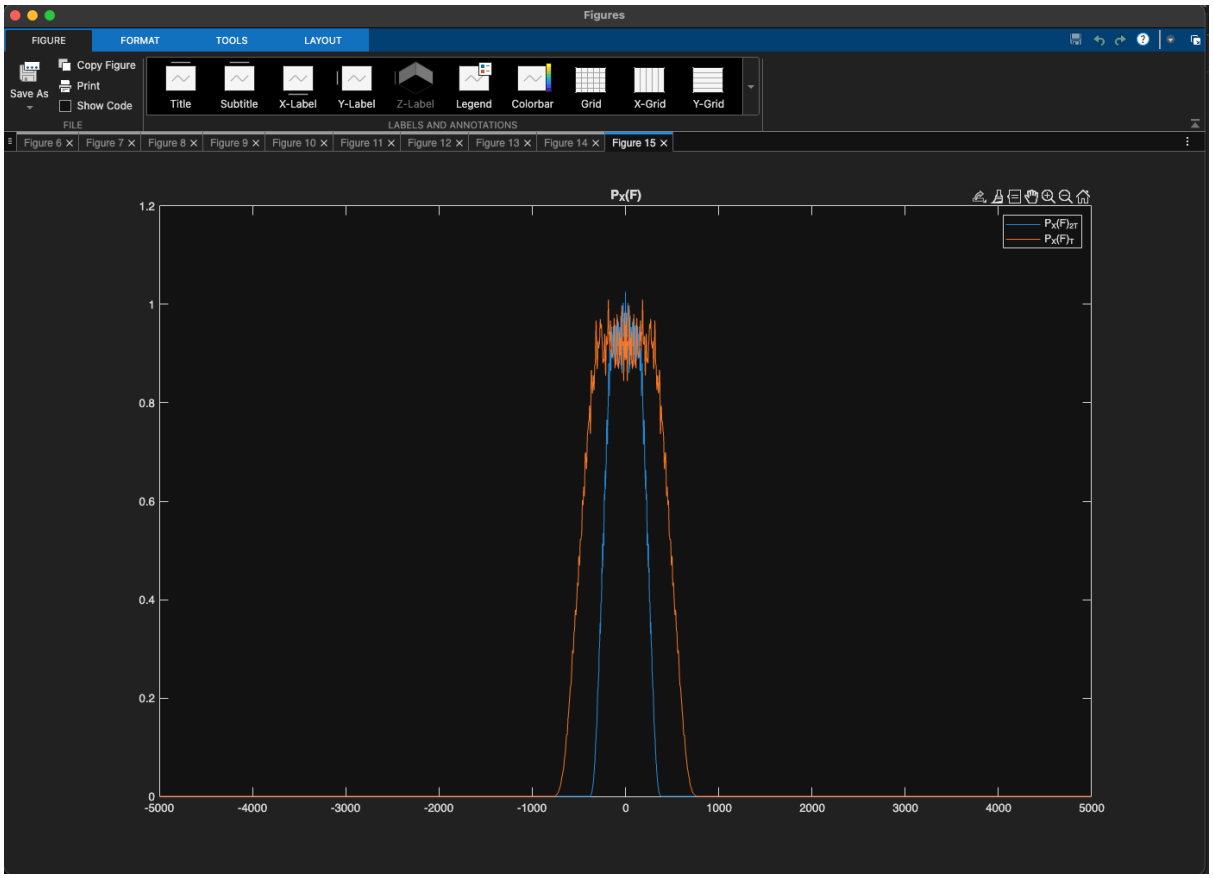
    % Create the periodogram
    X_F = fftshift(fft(X, Nf)*Ts);
    num = power(abs(X_F),2);
    T_total = X_time(end) - X_time(1);
    Px_F_total = Px_F_total + (num/T_total);
end
Px_F_estimated = Px_F_total/K;
Px_F_theoritical = 1/T*phi_Energy;

figure;
semilogy(f_axis,Px_F_estimated);
hold on;
semilogy(f_axis,Px_F_theoritical);
legend("Estimated","Theoritical");
title("P_x(F)");
xlim([-Fs/2 Fs/2]);
hold off;
```

(10) As you increase K and N, the approximation should improve. Does this happen in your experiments? If yes, can you explain why?

Solution:

As expected, increasing both K and N noticeably improved our approximation. By increasing the number of realizations K, we successfully averaged out the high variance and random noise inherent in individual periodograms, resulting in a much smoother curve. Similarly, increasing the symbol sequence length N generated a longer time-domain signal this inherently provided better frequency resolution during the Fourier Transform, allowing our experimental plot to trace the sharp drops and details of the theoretical spectrum much more accurately.



Code:

```
%----------------------3-----------------------
K_options = [1000; 50];
N_options = [200; 50];
figure;
for i=1:length(K_options)
    Px_F_total = 0;
    K_3 = K_options(i);
    N_3 = N_options(i);
    for j=1:K_3
        % 2-PAM bits generator
        b = (sign(randn(N_3, 1)) + 1)/2;
        x = bits_to_2PAM(b);

        % Upsample the 2-PAM bit vector
        upsample_x = (1/Ts)*upsample(x, over);
        upsample_x_time = 0:Ts:N_3*T-Ts;

        % Convolution of X and phi to compute the sum
        X = conv(upsample_x,phi)*Ts;
        X_time = upsample_x_time(1)+t(1):Ts:upsample_x_time(end)+t(end);

        % Create the periodogram
        X_F = fftshift(fft(X, Nf)*Ts);
        num = power(abs(X_F),2);
        T_total = X_time(end) - X_time(1);
        Px_F_total = Px_F_total + (num/T_total);
    end
    Px_F_estimated_3 = Px_F_total/K_3;
    Px_F_theoritical = 1/T*phi_Energy;

    subplot(1,2,i);
    semilogy(f_axis,Px_F_estimated_3);
    hold on;
    semilogy(f_axis,Px_F_theoritical);
    legend("Estimated","Theoritical");
    ylabel("P_x(F)");
    xlabel("Frequency");
    title(["K = ",K_3," N = ",N_3])
    xlim([-Fs/2 Fs/2]);
    hold off;
end
```

A.4

Using the mapping 00 →+3 01 →+1 11 →−1 10 →−3 construct the 4-PAM symbol sequence Xn, for n = 0,..., N 2 − 1. Note that, if the bits are equiprobable, then the symbols Xn are also equiprobable! Construct the waveform X(t)

Solution:

Firstly the function bits_to_4_PAM was created. The function is shown below:

/Users/kyrkos/Documents/MATLAB/MATLAB_LAB2/bits_to_4PAM.m

```
function X = bits_to_4PAM(b)

    b_pairs = reshape(b, 2, [])';

    % Αρχικοποίηση του διανύσματος των συμβόλων
    X = zeros(length(b)/2, 1);

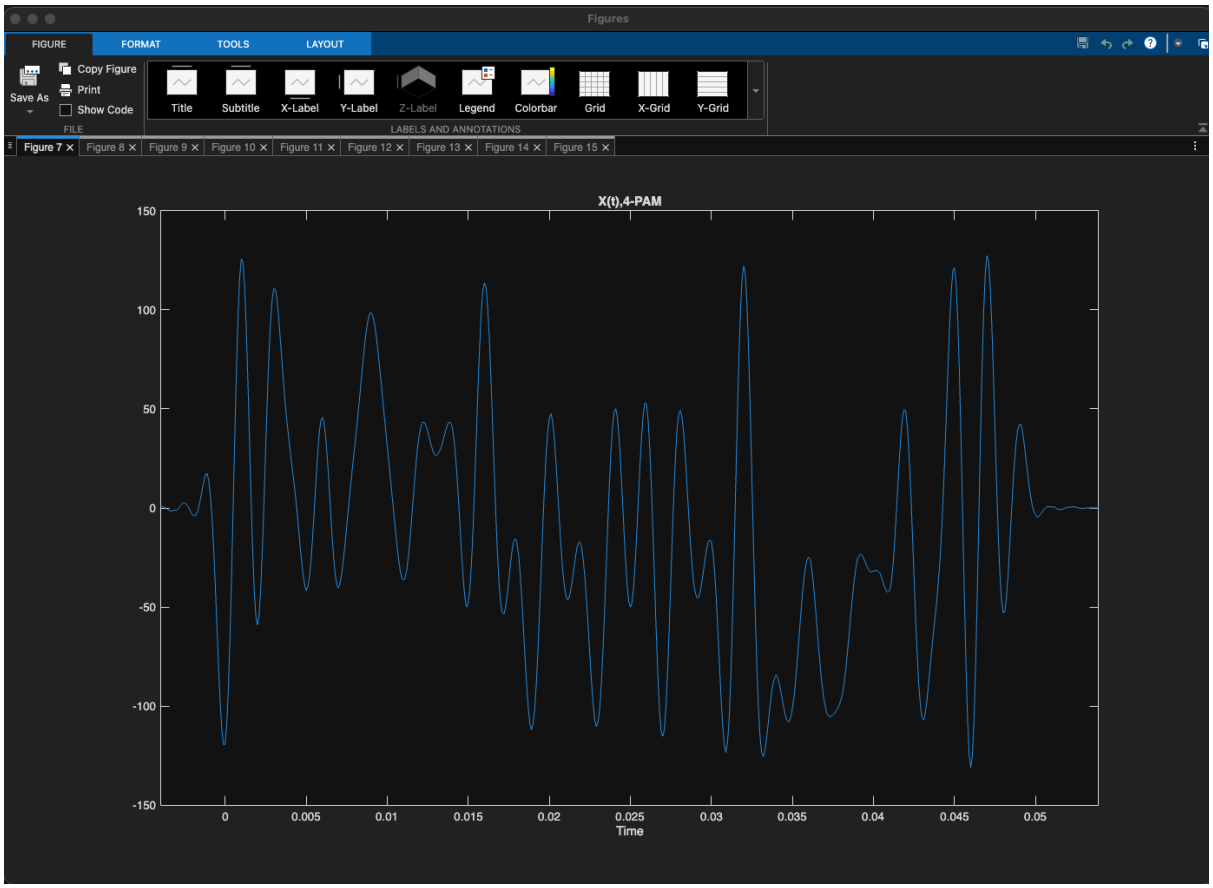
    % mapping
    idx_00 = (b_pairs(:,1) == 0) & (b_pairs(:,2) == 0);
    X(idx_00) = 3;

    idx_01 = (b_pairs(:,1) == 0) & (b_pairs(:,2) == 1);
    X(idx_01) = 1;

    idx_11 = (b_pairs(:,1) == 1) & (b_pairs(:,2) == 1);
    X(idx_11) = -1;

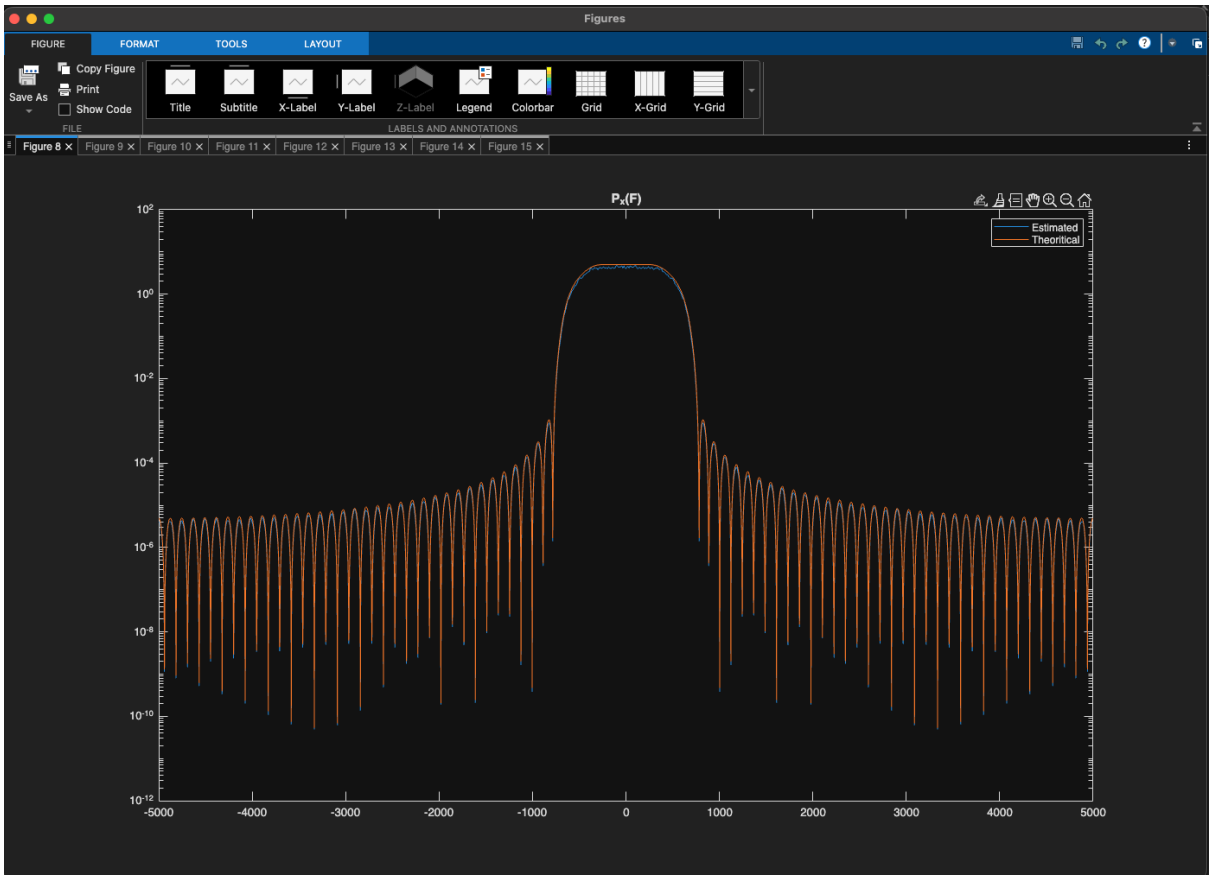
    idx_10 = (b_pairs(:,1) == 1) & (b_pairs(:,2) == 0);
    X(idx_10) = -3;
end
```

Then the same waveform as in A.2 was created. Only now we use 4 instead of 2-PAM. It is shown below:



(10) Compute the periodogram and estimate the power spectral density by averaging periodogram realizations of X(t). Plot the experimental and the theoretical power spectral density on the same semilogy. What do you observe?

Solution:



The use of 4-PAM halved our total number of transmitted symbols to N/2. After constructing the new X(t) waveforms using the same SRRC pulse , we computed and averaged their periodograms to estimate the experimental PSD.For the theoretical curve, we had to account for the new symbol amplitudes. Since all four symbols are equiprobable, we calculated the new variance as.

$$\sigma_X^2 = E(X_n^2) = \sum x^2 p_x = (1)^2\frac{1}{4} + (-1)^2\frac{1}{4} + (3)^2\frac{1}{4} + (-3)^2\frac{1}{4} = 5$$

We updated our theoretical formula to SX(F) = 5/T|Φ(F)|^2 and plotted both results on a semilogy graph. Just like in the 2-PAM case, we observed that the experimental PSD matched the theoretical curve perfectly, proving our simulation accurately models higher-order modulation.

Code:

```
%% A.4

% X(t)
b = (sign(randn(N, 1)) + 1)/2;
x_4PAM = bits_to_4PAM(b);

% Upsample x signal and time
x_up = (1/Ts)*upsample(x_4PAM, over);
x_up_time = 0:Ts:(N/2)*T-Ts;

% convolution
x = conv(x_up,phi)*Ts;
x_time = x_up_time(1)+t(1):Ts:x_up_time(end)+t(end);

figure;
plot(x_time,x);
title("X(t),4-PAM");
xlabel("Time");
xlim([x_time(1) x_time(end)]);

P_x_4PAM_total = 0;
for i=1:K
    % 4-PAM bits generator
    b = (sign(randn(N, 1)) + 1)/2;
    x = bits_to_4PAM(b);

    % Upsample the 2-PAM bit vector
    x_up = (1/Ts)*upsample(x, over);
    x_up_time = 0:Ts:(N/2)*T-Ts;

    % Convolution of X and phi to compute the sum
    X = conv(x_up,phi)*Ts;
    X_time = x_up_time(1)+t(1):Ts:x_up_time(end)+t(end);

    % Fourier transform of X(t)
    x_F_4PAM = fftshift(fft(X,Nf)*Ts);

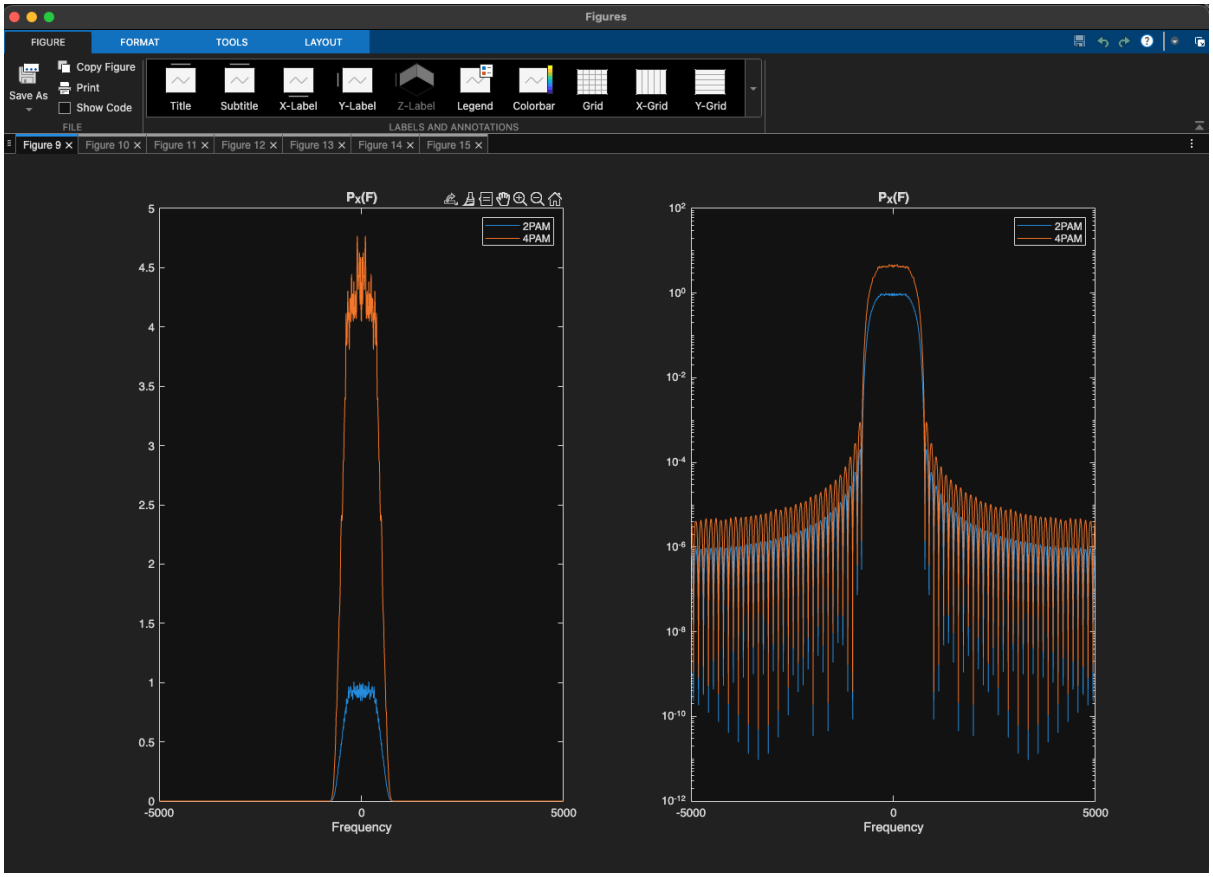
    % Create the periodogram
    T_total = X_time(end) - X_time(1);
    P_x_4PAM_num = power(abs(x_F_4PAM),2);
    P_x_4PAM_total = P_x_4PAM_total + (P_x_4PAM_num/T_total);
end

Px_F_estimated_4PAM = P_x_4PAM_total/K;
Px_F_theoritical_4PAM = 5/T*phi_Energy;

figure;
semilogy(f_axis,Px_F_estimated_4PAM);
hold on;
semilogy(f_axis,Px_F_theoritical_4PAM);
legend("Estimated","Theoritical");
title("P_x(F)");
xlim([-Fs/2 Fs/2]);
hold off;
```

(10) How does the power spectral density of this X(t) compare (in terms of bandwidth and peak magnitude) to the power spectral density of the X(t) in step A.2? Can you explain your observations?

Solution:



When comparing the two plotted spectra side-by-side, we observe that the bandwidth stays exactly the same, with both curves dropping to zero at the exact same frequencies. This makes sense because the bandwidth is dictated strictly by the SRRC shaping pulse and the symbol duration T which remained completely unchanged. However, the peak magnitude of the 4-PAM curve is noticeably higher specifically, 5 times larger than the 2-PAM curve. This happens because our new 4-PAM symbols carry much more energy this pushes the symbol variance σ^2X from 1 up to 5 which mathematically scales our entire theoretical PSD equation up by a factor of 5.

Code:

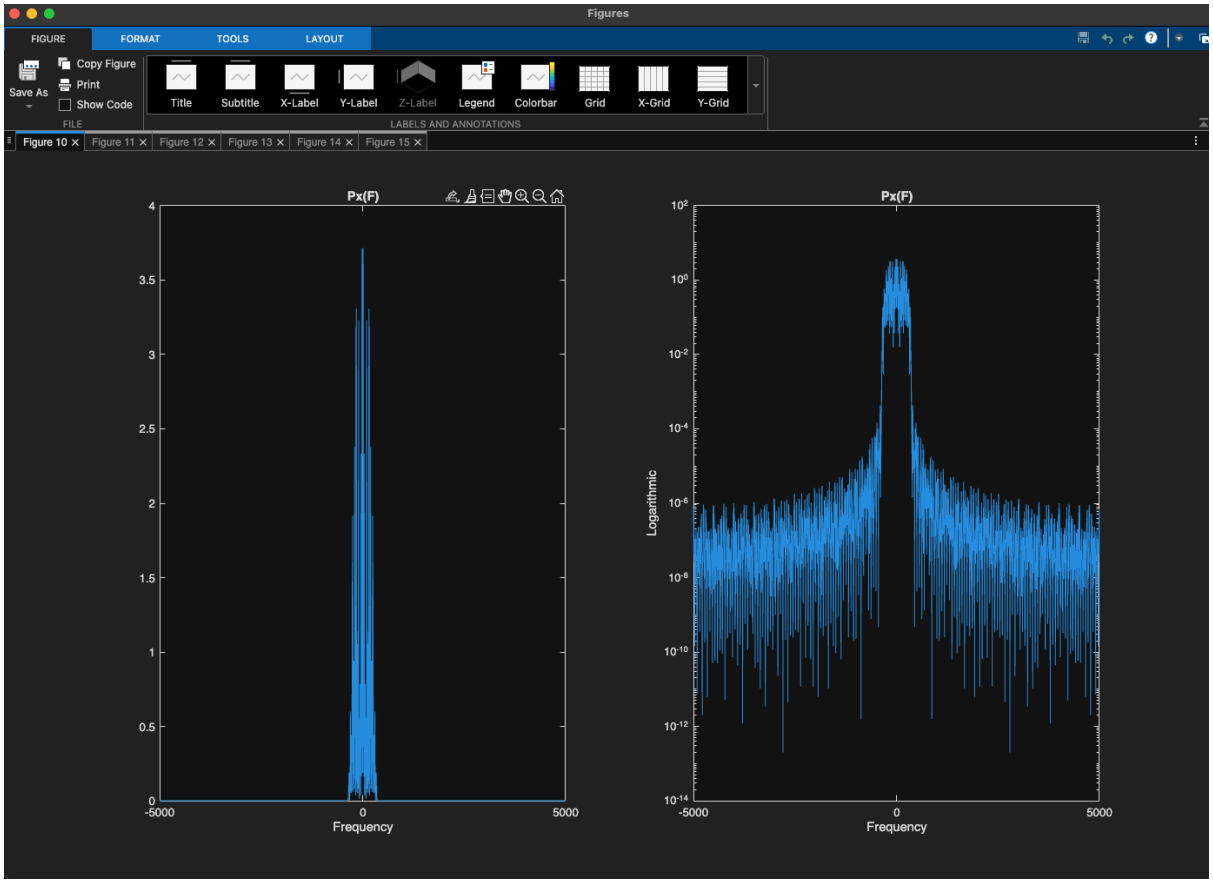
```
% plot vs seminology
figure;
subplot(1,2,1);
plot(f_axis, Px_F_estimated);
hold on;
plot(f_axis,Px_F_estimated_4PAM)
title("P_X(F)");
xlabel("Frequency");
legend("2PAM","4PAM");
xlim([-Fs/2 Fs/2]);
hold off;

subplot(1,2,2);
semilogy(f_axis,Px_F_estimated);
hold on;
semilogy(f_axis,Px_F_estimated_4PAM);
title("P_X(F)");
xlabel("Frequency");
legend("2PAM","4PAM");
xlim([-Fs/2 Fs/2]);
hold off;
```

A.5

(10) Repeat step A.3 by setting the symbol period to T′ = 2T (keep the sampling period Ts the same as in the previous steps; therefore, you must double the parameter over).

Solution:



As expected, the results are similar to A.3

Code:

```
%% A.5
T = 2*T;
over = 2*over;
% Create the new pulse
[phi, t] = srrc_pulse(T, over, A, a);

phi_F = fftshift(fft(phi, Nf)*Ts);
abs_phi_F = abs(phi_F);
phi_Energy = power(abs_phi_F,2);

% 2-PAM bits generator
b = (sign(randn(N, 1)) + 1)/2;
x = bits_to_2PAM(b);

% Upsample the 2-PAM bit vector
upsample_x = (1/Ts)*upsample(x, over);
upsample_x_time = 0:Ts:N*T-Ts;

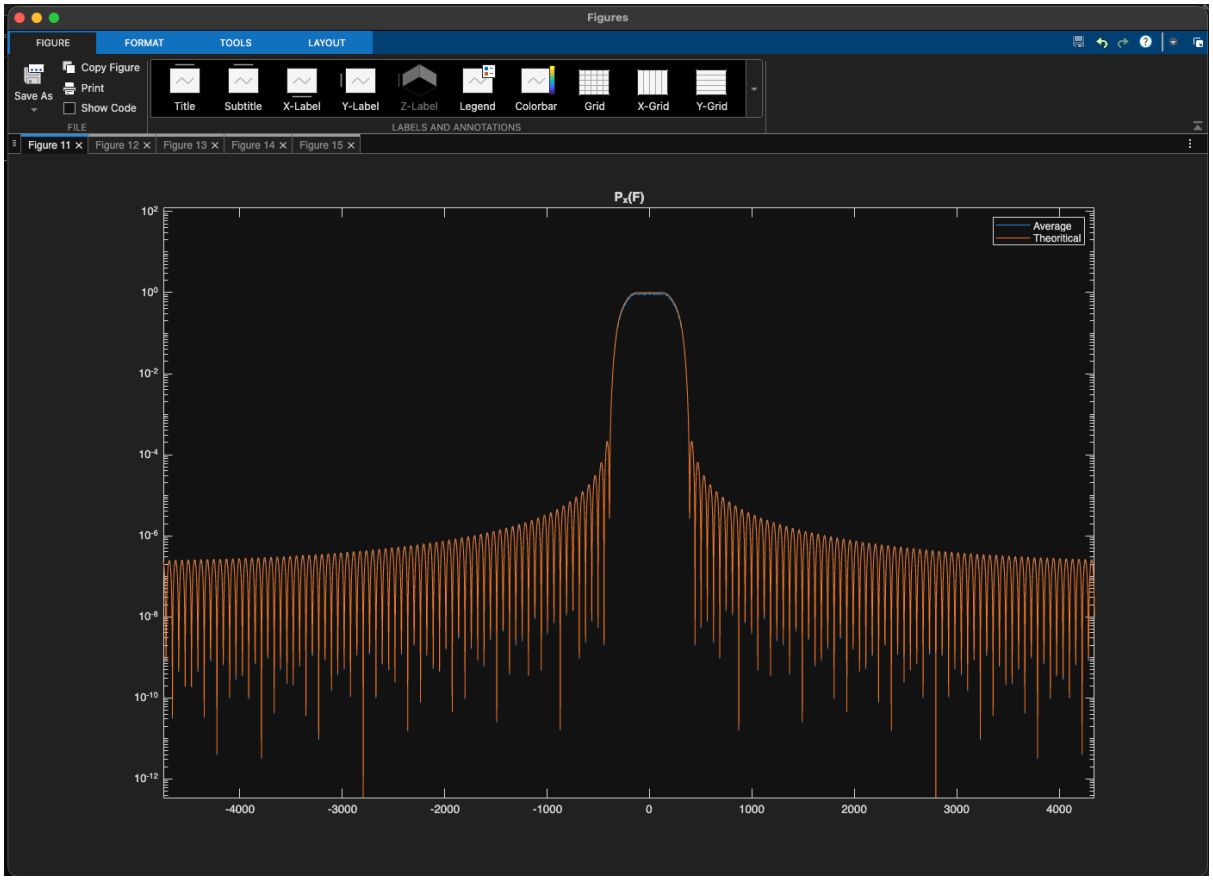
% Convolution of X and phi to compute the sum
X = conv(upsample_x,phi)*Ts;
X_time = upsample_x_time(1)+t(1):Ts:upsample_x_time(end)+t(end);


% periodogram
X_F = fftshift(fft(X, Nf)*Ts);
num = power(abs(X_F),2);
T_total = X_time(end) - X_time(1);
Px_F = num/T_total;

% Visualize Px(F) and RF Rx(F) using log scale
figure;
subplot(1,2,1);
plot(f_axis, Px_F);
title("Px(F)");
xlabel("Frequency");
xlim([-Fs/2 Fs/2]);

subplot(1,2,2);
semilogy(f_axis,Px_F);
title("Px(F)");
xlabel("Frequency");
ylabel("Logarithmic");
xlim([-Fs/2 Fs/2]);
```

Same with the graph comparing the theoretical and calculated power spectral density shown below:



Code:

```
Px_F_total_2T = 0;
K = 500;
for i=1:K
    % 2-PAM bits generator
    b = (sign(randn(N, 1)) + 1)/2;
    x = bits_to_2PAM(b);

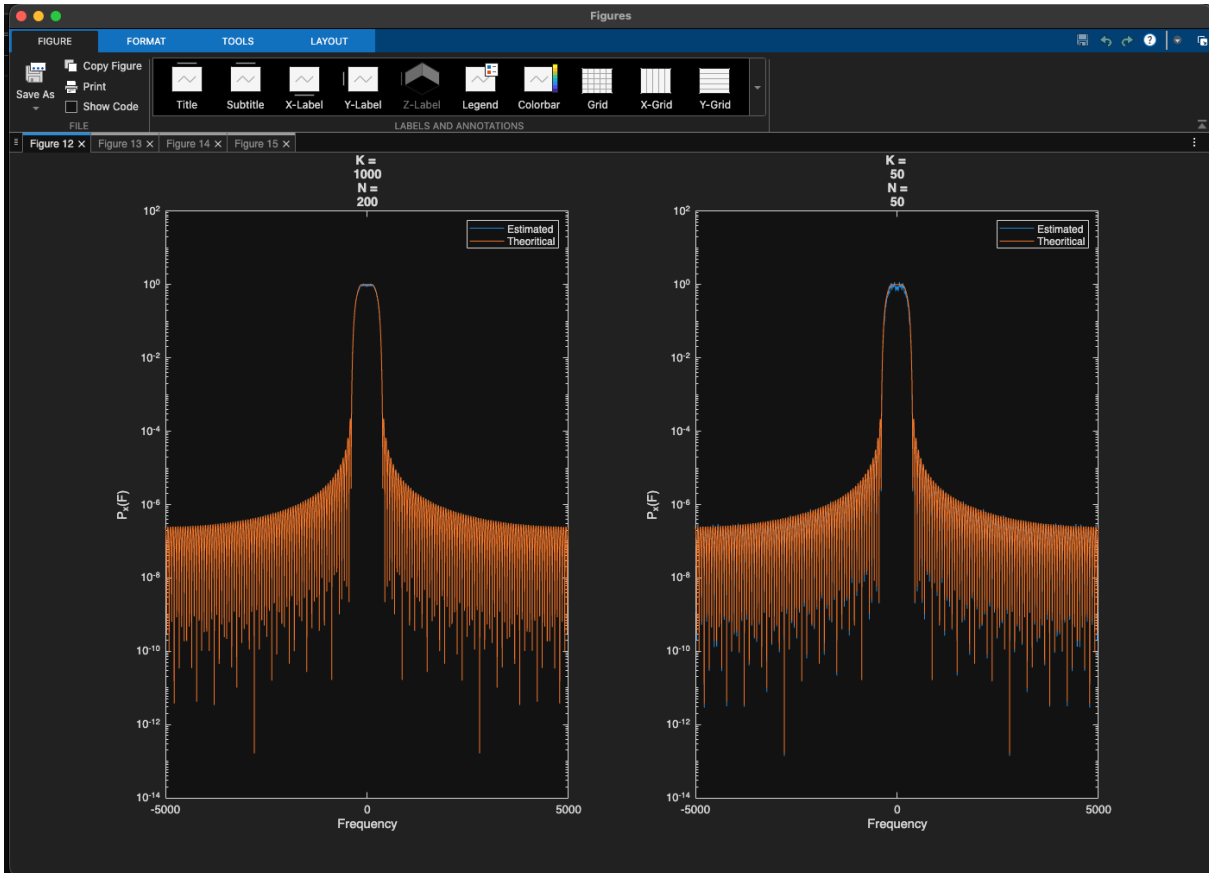
    % Upsample the 2-PAM bit vector
    upsample_x = (1/Ts)*upsample(x, over);

    % Convolution of X and phi to compute the sum
    X_t = conv(upsample_x,phi)*Ts;

    % Fourier Transform
    X_F = fftshift(fft(X_t, Nf)*Ts);
    % Create the periodogram
    num = power(abs(X_F),2);
    Px_F_total_2T = Px_F_total_2T + (num/T_total);
end
Px_F_estimated_2T = Px_F_total_2T/K;
Px_F_theoritical = 1/T*phi_Energy;

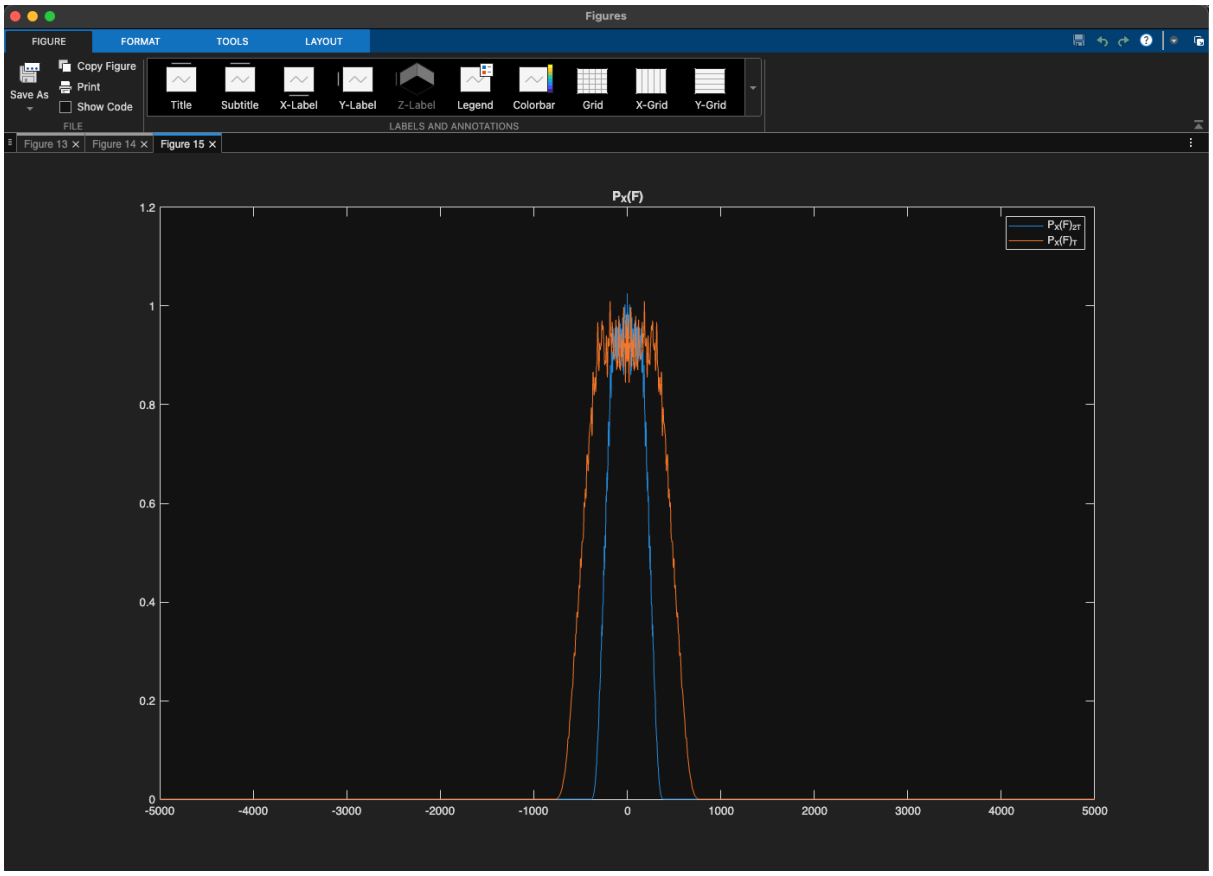
figure;
semilogy(f_axis,Px_F_estimated_2T,f_axis,Px_F_theoritical);
legend("Estimated","Theoritical");
title("P_x(F)");
xlim([-Fs/2 Fs/2]);
legend("Average","Theoritical");
```

As in A.3 we observe the same behavior when adjusting the K and N values. increasing both K and N noticeably improved our approximation. By increasing the number of realizations K, we successfully averaged out the high variance and random noise inherent in individual periodograms, resulting in a much smoother curve. Similarly, increasing the symbol sequence length N generated a longer time-domain signal this inherently provided better frequency resolution during the Fourier Transform, allowing our experimental plot to trace the sharp drops and details of the theoretical spectrum much more accurately.



(5) What do you observe about the bandwidth of the waveforms in this case compared to that of the waveforms in step A.3? Can you explain the phenomenon?

Solution:



In this step, we doubled the symbol period to T' = 2T while keeping the sampling rate constant (by also doubling the over factor). When comparing the resulting power spectral density to our original 2-PAM spectrum from step A.3, we observed that the bandwidth of the new waveform is exactly halved. This phenomenon is perfectly explained by the Time Scaling Property of the Fourier Transform. Because we are transmitting the symbols twice as slowly, the signal is essentially stretched in the time domain consequently, its spectrum is compressed in the frequency domain by the exact same factor, resulting in a narrower bandwidth that requires less space to transmit.

A.6

(2.5) If you wanted to transmit data as fast as possible while using the same available bandwidth, would you choose 2-PAM or 4-PAM, and why?

Solution:

To transmit data as fast as possible while strictly utilizing the same available bandwidth, I would definitely choose the 4-PAM system. As we observed experimentally in step A.4, the bandwidth of a PAM signal is determined entirely by the shaping pulse and the symbol period, and is completely unaffected by the number of amplitude levels. By upgrading from 2-PAM to 4-PAM, we increase the number of bits encoded per symbol from 1 bit to 2 bits. Since the symbol duration T remains identical, the 4-PAM system effectively doubles our data transmission rate without requiring a single extra Hertz of bandwidth. The only trade-off for this speed increase is the higher energy consumption required to transmit the larger amplitude levels.

(2.5) If the available bandwidth is very expensive, would you choose symbol period T or T′ =2T, and why?

Solution:

I would choose the longer symbol period T' = 2T. Based on the Time Scaling property of the Fourier Transform that we confirmed in A.5, transmitting symbols at a slower rate directly compresses the signal in the frequency domain. Specifically, using T' = 2T cuts the required bandwidth exactly in half compared to using the original period T. While this inherently means our overall data transmission speed will drop by 50%, it is the mathematically correct choice for minimizing our cost.

B: Stochastic processes

(5) Plot, on the same plot, 5 realizations of the process (only for this question, assume $F_0 = 10$ Hz, $T_s = 10^{-2}$ sec, and t axis = $[0 : T_s : 5/F_0]$).



Code:

/Users/kyrkos/Documents/MATLAB/MATLAB_LAB2/B.m

```
close all;

fo=10;
T=1/fo;
Ts=0.01;
t_axis = 0:Ts:5/fo;


N=1;
figure (1);
hold on;
for i = 1:5
    x = randn(N,1);
    phi = 2*pi*rand(1);
   Y = x.* cos(2*pi*fo*t_axis + phi);
   plot(t_axis,Y);
end

title("Y(t)");
hold off;
```

(10) Compute the quantities E[Y (t)] and $R_{YY}(t + \tau, t) = E[Y (t + \tau) Y (t)]$. What do you conclude?



Since X and Φ are independent, the expected value is the product of their individual expected values: E[Y(t)] = E[X * cos(2π * F_0 * t + Φ)] = E[X] * E[cos(2π * F_0 * t + Φ)]

Because X follows a standard normal distribution, its mean is zero (E[X] = 0). Therefore: E[Y(t)] = 0

The autocorrelation function is defined as: R_YY(t+τ, t) = E[Y(t+τ) * Y(t)]

Substituting the process equation: R_YY(t+τ, t) = E[X * cos(2π * F_0 * (t+τ) + Φ) * X * cos(2π * F_0 * t + Φ)]

R_YY(t+τ, t) = E[X^2] * E[cos(2π * F_0 * t + 2π * F_0 * τ + Φ) * cos(2π * F_0 * t + Φ)]

Since X ~ N(0, 1), its second moment is E[X^2] = Var(X) + (E[X])^2 = 1 + 0 = 1.

Using the trigonometric identity cos(A)cos(B) = (1/2) * [cos(A-B) + cos(A+B)], the expression becomes: R_YY(t+τ, t) = 1 * E[(1/2)*cos(2π * F_0 * τ) + (1/2)*cos(4π * F_0 * t + 2π * F_0 * τ + 2Φ)]

$R_{YY}(t+\tau, t) = (1/2)*\cos(2\pi * F_0 * \tau) + (1/2)*E[\cos(4\pi * F_0 * t + 2\pi * F_0 * \tau + 2\Phi)]$

The expected value of the second cosine term over the uniform phase Φ in $[0, 2\pi)$ evaluates to zero, because integrating a cosine over two full periods yields zero. Thus, we are left with: $R_{YY}(t+\tau, t) = (1/2) * \cos(2\pi * F_0 * \tau)$

(5) Compute the power spectral density $S_Y(F)$.

Solution:

$S_Y(F) = \text{Integral from } -\infty \text{ to } +\infty [R_{YY}(\tau) * e^{(-j2\pi f\tau)} d\tau]$

So: $S_Y(F) = F[R_{YY}] = F[(1/2) * \cos(2\pi * f0 * \tau)]$

From Euler's formula: $\cos(2\pi f0\tau) = (1/2) * [e^{(j2\pi f0\tau)} + e^{(-j2\pi f0\tau)}]$

$F\{ \cos(2\pi f0\tau) \} = (1/2) * [F\{ e^{(j2\pi f0\tau)} \} + F\{ e^{(-j2\pi f0\tau)} \}]$

$F\{ e^{(j2\pi f0\tau)} \} = \delta(f - f0)$ $F\{ e^{(-j2\pi f0\tau)} \} = \delta(f + f0)$

So: $F\{ \cos(2\pi f0\tau) \} = (1/2) * [\delta(f - f0) + \delta(f + f0)]$

Then the power spectral density is:

$S_Y(F) = F[R_{YY}]$ $S_Y(F) = (1/2) * F\{ \cos(2\pi f0\tau) \}$ $S_Y(F) = (1/4) * [\delta(f - f0) + \delta(f + f0)]$

For F0 = 10: $S_Y(F) = (1/4) * [\delta(f - 10) + \delta(f + 10)]$

Basically, since our original signal Y(t) is just a pure cosine wave oscillating at one specific frequency, it doesn't contain a messy mix of different high or low frequencies. Because of this, 100% of the signal's power is concentrated exactly at that one spot. All the signal's energy is packed tightly at +f0 and -f0 instead of being spread out everywhere.